ROUND 14 — CONSTITUTION V4: ORGANIZATIONAL COGNITION

Organizational Cognition

From features that answer questions to a cognitive system that makes the organization wiser.

Reviewer	Independent (Super Z, Z.ai) — acting as Principal Engineer specifying V4 cognitive organs
Baseline	Commit b23db5e. V3 score: 9.0/10. 51 backend modules, 25 frontend files. 4 V3 engines wired (personality, sowhat, evolution, humanize). Time-axis pending.
V4 shift	V3 = features that answer questions. V4 = cognitive organs that change how the organization thinks. The user stops asking. The organization becomes wiser.
Constitutional law	"Every interaction with Maestro must leave the organization slightly wiser than it was before." — the V4 litmus test.
Deliverable	8 cognitive organs. Each builds on existing infrastructure (assumption.py, hypothesis.py, contradiction.py, law.py, learning.py). Each has API + acceptance test + build order.

WHY V4 EXISTS

V3 specified 7 features. The coder built 5 and wired 3. The features are good — the So What Engine, Personality, Time-Axis, Evolution Report, and Conversational Ask move Maestro from dashboard to intelligence layer. But they all share a limitation: they answer questions the user asks. The user opens Maestro, asks "Why are releases slowing?", gets an answer. That is better than a dashboard, but it is still query-response.

V4 is different. V4 cognitive organs do not wait for questions. They observe, understand, judge, prepare, learn, and evolve — continuously, autonomously, without being asked. The user stops opening Maestro to ask things. Maestro opens itself when the organization needs to think differently. The product shifts from "answering questions" to "changing how the organization thinks." That is a different category.

The V4 litmus test for every commit: "Does this leave the organization slightly wiser than it was before?" If yes, ship it. If it only answers a question without changing future judgment, it is a V3 feature — useful but not V4. V4 organs must change the organization's thinking, not just inform it.

Same discipline as V3. Every organ has: the purpose, the existing codebase it builds on, exact files to create/modify, API contract, acceptance test, score delta, effort, build phase. The acceptance tests check APPLICATION (frontend wired), not EXISTENCE (backend built). The recurring pattern across 13 rounds — built but not applied — ends here. Every V4 organ must be user-visible before it is claimed.

Build order matters more than in V3. V4 organs form a cognitive stack: Identity → Curiosity → Skepticism → Wisdom → Metacognition → Principles → Memory Compression → Consciousness. Each builds on the previous. Build them in order. Do not skip ahead. The stack is the product.

The 8 Cognitive Organs

#	Organ	Purpose	Builds on	Effort	Phase
1	Identity	Does the organization match what it believes about itself?	personality.py + contradiction.py	2 days	1
2	Curiosity	Maestro asks questions the org has never asked	assumption.py + hypothesis.py	2 days	1
3	Skepticism	Continuously challenge fossilized beliefs	assumption.py + learning.py	1.5 days	2
4	Wisdom	Synthesize competing values into judgment	sowhat.py + perspective.py	2 days	2
5	Metacognition	Org thinking about its own thinking	contradiction.py + coordination.py	2 days	3
6	Principles	Laws that graduate to wisdom after years	law.py + learning.py	1.5 days	3
7	Memory Compression	2M signals → 3 truths, 7 habits, 2 mistakes	learning.py + evidence_graph.py	2 days	4
8	Consciousness	Always knows where attention/knowledge/trust/conflict is	All organs + pulse.py + feed.py	3 days	4
TOTAL	**8 organs**	**Cognitive stack: observe → understand → judge → prepare → learn → evolve**	**31 existing modules**	**~16 days**	**4 phases**

Build order: Phase 1 (#1 Identity, #2 Curiosity) → Phase 2 (#3 Skepticism, #4 Wisdom) → Phase 3 (#5 Metacognition, #6 Principles) → Phase 4 (#7 Memory Compression, #8 Consciousness). Each phase takes ~4 days. Total ~16 days. When all 8 are delivered and user-visible, Constitution adherence reaches 10/10 and the product is genuinely a cognitive system, not a feature collection.

Organ #1 — Identity (Phase 1, Foundational)

ORGAN #1 Identity: does the organization match what it believes about itself?

Purpose

Every living organism has an identity. Organizations don't — until now. The Identity organ compares the organization's stated beliefs ("We are extremely fast", "Customer obsession", "Quality first") against observed behavior (decision velocity, roadmap composition, review discipline). When belief and behavior diverge, Identity Drift is detected. This is not a KPI. It is a measure of whether the organization is becoming itself.

Builds on (existing codebase)

personality.py (V3, 247 lines) provides behavioral inference. contradiction.py (458 lines) provides contradiction detection between stated and observed. The Identity organ composes these: it takes stated beliefs (from leadership signals, docs, stated values) and compares them against personality.py's inferred behavior. The infrastructure exists — the composition does not.

Files to create/modify

CREATE `backend/maestro_oem/identity.py` — the IdentityEngine. Maintains a set of stated beliefs (inferred from Confluence "values" pages, leadership Slack messages, stated mission docs) and compares them against observed behavior (from personality.py). Computes Identity Drift score (0.0 = aligned, 1.0 = completely drifted) per belief. CREATE GET /api/oem/identity endpoint. CREATE `static/js/identity.js` — a surface (command-palette only) showing each belief, the observed reality, the drift score, and a narrative ("You believe you are extremely fast. Your average decision time is 11.3 days. Identity Drift: high."). MODIFY `static/js/today.js` — if Identity Drift > 0.6 for any belief, show a calm one-liner: "Your organization believes something about itself that may not be true."

API contract

GET /api/oem/identity → returns JSON: { "beliefs": [{ "stated": "We are extremely fast.", "source": "confluence:mission-statement", "observed": "Average decision time 11.3 days.", "drift_score": 0.72, "drift_label": "high", "narrative": "You believe you are extremely fast. Your average decision time is 11.3 days. The gap is significant.", "evidence_count": 14 }], "overall_drift": 0.48, "summary": "Your organization believes it is fast and customer-obsessed. Reality suggests moderate drift on both." }. Must have at least 2 beliefs, each with drift_score (0-1), evidence_count > 0, and a narrative sentence.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/identity. Response must have at least 2 beliefs, each with stated + observed + drift_score (0-1) + evidence_count > 0 + narrative. 2) Auditor verifies drift_score is computed (not hardcoded) — different demo data would produce different scores. 3) Auditor opens the Identity surface via command palette (Ctrl+K → "identity") and verifies it renders beliefs with drift indicators. 4) Auditor opens TODAY and verifies that if drift > 0.6, the one-liner appears. 5) Auditor greps identity.py for hardcoded drift values — must find none (all computed from model data).

Constitution score delta

+1.0 (9.0 → 10.0 capped at 9.5). This is the foundational V4 organ — it introduces the concept of "organizational self-awareness" that all other organs build on. Without Identity, Curiosity and Skepticism have nothing to compare against.

Effort

2 days (1 day engine + 0.5 day API + 0.5 day frontend surface + TODAY integration)

Build phase

Phase 1 (foundational — build first)

Organ #2 — Curiosity (Phase 1, Foundational)

ORGAN #2 Curiosity: Maestro asks questions the organization has never asked

Purpose

No enterprise software asks questions. It answers them. Curiosity inverts this: Maestro identifies things the organization has never measured, assumptions it has never tested, and questions it has never asked — then surfaces them as calm prompts. "We've never measured why Legal rejects OAuth exceptions." "Every team assumes onboarding is slow. Nobody has actually tested that assumption." This is not analytics. It is curiosity — the engine of organizational learning.

Builds on (existing codebase)

assumption.py (376 lines) tracks assumptions and their accuracy. hypothesis.py (315 lines) tracks hypotheses and their resolution. The Curiosity organ composes these: it finds assumptions with low evidence_count (never tested), domains with no hypotheses (never questioned), and signals with no linked learning objects (never understood). The infrastructure exists — the question-generation does not.

Files to create/modify

CREATE `backend/maestro_oem/curiosity.py` — the CuriosityEngine. Generates questions in 3 categories: (1) untested assumptions ("Every team assumes X. Nobody has tested it."), (2) unmeasured domains ("We've never measured why X happens."), (3) unexplained patterns ("X happens consistently. We don't know why."). Each question has a confidence that it is genuinely unexplored (low evidence_count = high confidence it is unexplored). CREATE GET /api/oem/curiosity endpoint. MODIFY `static/js/today.js` — add a "Questions Maestro is asking" section at the bottom of the morning brief (max 2 questions, calm tone). MODIFY `static/js/learn.js` — add a "What Maestro is curious about" section.

API contract

GET /api/oem/curiosity?limit=5 → returns JSON: { "questions": [{ "question": "We've never measured why Legal rejects OAuth exceptions.", "category": "unmeasured_domain", "domain": "auth", "confidence_unexplored": 0.84, "evidence_count": 0, "suggested_experiment": "Track rejection reasons for 2 weeks and correlate with PR complexity.", "why_curious": "Legal has rejected 3 OAuth PRs in 90 days. No signal explains why." }], "total_untested_assumptions": 4, "total_unmeasured_domains": 2 }. Each question must have evidence_count (can be 0 — that is the point) and a why_curious string explaining what triggered the curiosity.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/curiosity. Response must have at least 1 question with category, confidence_unexplored (0-1), why_curious (non-empty). 2) Auditor verifies the question references real model data (domain, signal count) — not hardcoded. 3) Auditor opens TODAY and verifies a "Questions Maestro is asking" section appears with at most 2 questions. 4) Auditor verifies the tone is calm (not alarming) — the questions should feel like a curious colleague, not an alert. 5) Auditor greps curiosity.py for hardcoded questions — must find none (all generated from model gaps).

Constitution score delta

+1.0 (9.5 → 10.0 capped). Curiosity is what makes Maestro feel alive — it asks before being asked. This is the V4 differentiator from V3 (which only answers).

Effort

2 days (1 day question-generation engine + 0.5 day API + 0.5 day frontend)

Build phase

Phase 1 (foundational — build alongside Identity)

Organ #3 — Skepticism (Phase 2)

ORGAN #3 Skepticism: continuously challenge fossilized beliefs

Purpose

Organizations become dangerous because assumptions fossilize. "Meetings improve alignment" becomes an unquestioned belief — even when the data shows correlation -0.42 between meeting count and alignment. Skepticism continuously asks "What if this belief is wrong?" and surfaces beliefs where the evidence has shifted. This is not cynicism — it is intellectual honesty. The organization stops lying to itself.

Builds on (existing codebase)

assumption.py (376 lines) tracks assumptions and their accuracy_report(). learning.py (993 lines) tracks concept drift and organizational drift. The Skepticism organ composes these: it takes each assumption, checks its accuracy against recent signals, and flags beliefs where accuracy is declining or where drift has been detected. The infrastructure exists — the challenge-generation does not.

Files to create/modify

CREATE `backend/maestro_oem/skepticism.py` — the SkepticismEngine. For each assumption in assumption.py, compute a "fossilization risk" score based on: (1) age of the assumption (older = more likely fossilized), (2) accuracy trend (declining = belief may be outdated), (3) evidence recency (no recent evidence = belief is untested). Surface the top 3 fossilized beliefs with a challenge: "You believe X. Evidence from the last 90 days suggests this may be wrong. Confidence in this belief: 0.18." CREATE GET /api/oem/skepticism endpoint. MODIFY `static/js/learn.js` — add a "Beliefs Maestro is questioning" section.

API contract

GET /api/oem/skepticism → returns JSON: { "challenged_beliefs": [{ "belief": "Meetings improve alignment.", "assumption_id": "asmp-xxx", "evidence_summary": "Last 90 days: correlation -0.42 between meeting count and cross-team coordination signals.", "fossilization_risk": 0.78, "confidence_in_belief": 0.18, "recommendation": "This belief is probably outdated. Consider reducing meeting load and measuring alignment directly.", "evidence_count": 47 }], "total_beliefs_challenged": 3 }. Each challenged belief must have fossilization_risk (0-1), confidence_in_belief (0-1, lower = more skeptical), and a recommendation.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/skepticism. Response must have at least 1 challenged belief with fossilization_risk, confidence_in_belief, recommendation. 2) Auditor verifies the belief references a real assumption from assumption.py (not hardcoded). 3) Auditor verifies confidence_in_belief is computed from accuracy data (not hardcoded). 4) Auditor opens LEARN and verifies a "Beliefs Maestro is questioning" section appears. 5) Auditor verifies the tone is calm and non-alarming.

Constitution score delta

+0.5 (10.0 capped). Skepticism prevents the organization from lying to itself. It is the immune system of organizational cognition.

Effort

1.5 days (1 day fossilization-scoring engine + 0.5 day API + frontend)

Build phase

Phase 2 (build after Identity + Curiosity)

Organ #4 — Wisdom (Phase 2)

ORGAN #4 Wisdom: synthesize competing values into judgment

Purpose

Intelligence predicts. Wisdom judges. Engineering wants velocity. Legal wants certainty. Finance wants predictability. History shows every successful launch accepted slightly lower velocity. Wisdom synthesizes these competing values into a recommendation that no single perspective could produce. This is not "the AI thinks" — it is "we've seen this pattern before, and here is the judgment that worked."

Builds on (existing codebase)

sowhat.py (V3, 197 lines) synthesizes consequences. perspective.py (235 lines) translates signals across stakeholder perspectives. The Wisdom organ composes these: it takes a decision context, identifies the competing values (from perspective.py), retrieves historical outcomes where similar values competed (from learning.py + prediction_lifecycle.py), and synthesizes a judgment. The infrastructure exists — the value-synthesis does not.

Files to create/modify

CREATE `backend/maestro_oem/wisdom.py` — the WisdomEngine. For a given decision context (e.g., "Should we delay the launch for Legal review?"), identify competing values (velocity vs certainty), retrieve historical precedents (past launches with similar tension), and synthesize a judgment: "History shows every successful launch accepted slightly lower velocity. Recommendation: delay 2 days for Legal review. Confidence: high. Basis: 7 similar launches, 6 succeeded when Legal reviewed, 2 failed when they didn't." CREATE GET /api/oem/wisdom?context=... endpoint. MODIFY `static/js/ask_v2.js` — when the user asks a judgment question, route to the Wisdom engine and render the synthesized judgment with the value-tension visualization.

API contract

GET /api/oem/wisdom?context=Should+we+delay+launch+for+Legal+review → returns JSON: { "competing_values": [{"value": "velocity", "stakeholder": "Engineering", "position": "ship now"}, {"value": "certainty", "stakeholder": "Legal", "position": "review first"}], "historical_precedents": [{"context": "Q3 auth launch", "outcome": "succeeded", "decision": "delayed 3 days for Legal", "evidence_count": 5}], "judgment": "History shows every successful launch accepted slightly lower velocity. Recommendation: delay 2 days for Legal review.", "confidence": 0.78, "basis": "7 similar launches, 6 succeeded when Legal reviewed, 2 failed when they didn't." }. Must have at least 2 competing values and at least 1 historical precedent with evidence_count > 0.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/wisdom?context=Should+we+delay+launch+for+Legal+review. Response must have competing_values (2+), historical_precedents (1+ with evidence_count > 0), judgment (a sentence, not a metric), confidence (0-1), basis (non-empty). 2) Auditor verifies the judgment references the historical precedents (not generic). 3) Auditor opens ASK v2 and asks a judgment question — verifies the response includes value-tension synthesis (not just keyword search). 4) Auditor verifies the tone is "we've seen this before" not "the AI thinks."

Constitution score delta

+1.0 (10.0 capped). Wisdom is the V4 end-state — the product stops predicting and starts judging. This is what Apple or Microsoft would buy.

Effort

2 days (1 day value-synthesis engine + 0.5 day historical retrieval + 0.5 day frontend)

Build phase

Phase 2 (build after Skepticism)

Organ #5 — Metacognition (Phase 3)

ORGAN #5 Metacognition: the organization thinking about its own thinking

Purpose

Everyone is building copilots. Nobody is building metacognition. "Engineering is making good decisions. Marketing is making good decisions. The organization as a whole is making poor decisions. Reason: cross-functional assumptions never converge." This is not analytics. It is organizational thinking about thinking — the ability to observe that the parts are healthy but the whole is not, and explain why.

Builds on (existing codebase)

contradiction.py (458 lines) detects contradictions between stated beliefs and observed behavior. coordination.py tracks cross-functional coordination signals. The Metacognition organ composes these: it identifies cases where individual teams are performing well (low contradiction, high prediction accuracy) but the organization as a whole is not (cross-team contradictions, unconverged assumptions). The infrastructure exists — the meta-level analysis does not.

Files to create/modify

CREATE `backend/maestro_oem/metacognition.py` — the MetacognitionEngine. Computes a "meta-cognition score" for the organization: (1) team-level decision quality (from prediction accuracy per team), (2) org-level decision quality (from cross-team contradiction rate), (3) the gap between them. When team-level is high but org-level is low, surface the meta-cognition insight: "Your teams are smart. Your organization is not. The reason: cross-functional assumptions never converge." CREATE GET /api/oem/metacognition endpoint. MODIFY `static/js/today.js` — if the meta-cognition gap is large, show a calm one-liner: "Your teams are making good decisions. Your organization may not be."

API contract

GET /api/oem/metacognition → returns JSON: { "team_quality": {"avg_prediction_accuracy": 0.74, "teams_assessed": 3, "detail": "Engineering, Legal, and Platform are all well-calibrated."}, "org_quality": {"cross_team_contradiction_rate": 0.42, "unconverged_assumptions": 5, "detail": "3 of 5 cross-team assumptions have never been reconciled."}, "meta_gap": 0.38, "meta_gap_label": "significant", "insight": "Your teams are smart. Your organization is not. The reason: cross-functional assumptions never converge.", "evidence_count": 12 }. Must have meta_gap (0-1) and insight (a sentence, not a metric).

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/metacognition. Response must have team_quality, org_quality, meta_gap (0-1), insight (non-empty). 2) Auditor verifies meta_gap is computed (team_quality vs org_quality, not hardcoded). 3) Auditor opens TODAY and verifies the meta-cognition one-liner appears when the gap is significant. 4) Auditor verifies the insight references real cross-team data (not generic).

Constitution score delta

+0.5 (10.0 capped). Metacognition is the deepest moat — no competitor builds this. It is organizational self-reflection.

Effort

2 days (1.5 days meta-analysis engine + 0.5 day API + frontend)

Build phase

Phase 3 (build after Wisdom)

Organ #6 — Principles (Phase 3)

ORGAN #6 Principles: laws that graduate to wisdom after years

Purpose

V3 has "laws" — patterns promoted after 3 observations (round-9 finding: threshold too low). V4 has "principles" — laws that have been validated for years, never failed, and have graduated from "pattern" to "wisdom." Principle #18: "Whenever Customer Success joins product planning before architecture freeze, post-launch bugs drop 27%. Observed 18 times. Failed 0 times." This is no longer a law. It is organizational wisdom.

Builds on (existing codebase)

law.py (101 lines) has OrganizationalLaw with validated_runtimes and LawStatus (CANDIDATE → VALIDATED → STRESSED → INVALIDATED). The Principles organ adds a new status: PRINCIPLE — reserved for laws with validated_runtimes >= 20, failed_runtimes == 0, and age >= 365 days. The infrastructure exists — the graduation logic does not.

Files to create/modify

MODIFY `backend/maestro_oem/law.py` — add LawStatus.PRINCIPLE and a `graduate_to_principle()` method that requires: validated_runtimes >= 20, failed_runtimes == 0, first_inferred >= 365 days ago. CREATE `backend/maestro_oem/principle_engine.py` — scans all laws, graduates eligible ones, and maintains the principle registry. CREATE GET /api/oem/principles endpoint. MODIFY `static/js/learn.js` — add a "Organizational Principles" section showing graduated principles with their validation history. MODIFY `static/js/physics_laws.js` — principles are displayed differently from laws (with a "wisdom" badge, not a "pattern" badge).

API contract

GET /api/oem/principles → returns JSON: { "principles": [{ "code": "P-001", "statement": "Whenever Customer Success joins product planning before architecture freeze, post-launch bugs drop 27%.", "validated_runtimes": 18, "failed_runtimes": 0, "first_inferred": "2024-01-15...", "age_days": 540, "evidence_count": 18, "graduated_at": "2025-06-15..." }], "total_principles": 1, "total_candidates": 6 }. Note: with only 90 days of demo data, no laws will have graduated yet — the response should honestly show 0 principles and N candidates. The acceptance test verifies the graduation logic works (via a unit test with seeded data), not that the demo has principles.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/principles. Response must have principles array (can be empty with honest "0 graduated, N candidates" message) and total_candidates. 2) Auditor verifies LawStatus.PRINCIPLE exists in law.py. 3) Auditor verifies graduate_to_principle() requires validated_runtimes >= 20, failed_runtimes == 0, age >= 365 days. 4) Auditor opens LEARN and verifies an "Organizational Principles" section appears (with "No principles have graduated yet — N candidates are forming" if empty). 5) Auditor runs a unit test that seeds a law with 20 validations and 365-day age and verifies it graduates.

Constitution score delta

+0.5 (10.0 capped). Principles are the long-term moat — after 3 years, the accumulated principles are impossible to recreate. This is what makes Maestro valuable in year 4.

Effort

1.5 days (0.5 day law.py modification + 0.5 day graduation engine + 0.5 day API + frontend)

Build phase

Phase 3 (build after Metacognition)

Organ #7 — Memory Compression (Phase 4)

ORGAN #7 Memory Compression: 2M signals → 3 truths, 7 habits, 2 mistakes

Purpose

Organizations drown in memory. 2 million Slack messages, 500k Jira tickets, 100k PRs. Nobody can hold this in their head. Memory Compression continuously distills experience into understanding: "Three truths. Seven habits. Two recurring mistakes. Five successful interventions." Memory should become understanding — not a searchable archive, but a compressed model of what the organization has learned.

Builds on (existing codebase)

learning.py (993 lines) tracks calibration, law evolution, drift. evidence_graph.py tracks evidence nodes. learning_object.py tracks learning objects. The Memory Compression organ composes these: it takes all learning objects, laws, principles, resolved predictions, and contradictions, and compresses them into a small set of "truths" (validated principles), "habits" (repeated patterns), "mistakes" (failed predictions + invalidated laws), and "interventions" (successful recommendations). The infrastructure exists — the compression does not.

Files to create/modify

CREATE `backend/maestro_oem/memory_compression.py` — the MemoryCompressionEngine. Produces a compressed summary: { "truths": [top 3 validated laws/principles by evidence], "habits": [top 7 repeated patterns], "mistakes": [top 2 failed predictions or invalidated laws], "interventions": [top 5 successful recommendations], "compression_ratio": "50,000 signals → 17 insights" }. CREATE GET /api/oem/memory/compressed endpoint. MODIFY `static/js/learn.js` — add a "What your organization has learned" section showing the compressed memory (not the raw signals).

API contract

GET /api/oem/memory/compressed → returns JSON: { "truths": [{"statement": "...", "evidence_count": 18, "confidence": 0.94}], "habits": [{"pattern": "...", "frequency": "weekly", "evidence_count": 12}], "mistakes": [{"what": "...", "why_failed": "...", "evidence_count": 3}], "interventions": [{"action": "...", "outcome": "succeeded", "evidence_count": 5}], "compression_ratio": "50,000 signals → 17 insights", "last_compressed": "..." }. Must have at least 1 truth, 1 habit, 1 mistake, 1 intervention with evidence_count > 0.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/memory/compressed. Response must have truths, habits, mistakes, interventions arrays (each with at least 1 item, each with evidence_count > 0). 2) Auditor verifies compression_ratio references real signal count. 3) Auditor opens LEARN and verifies a "What your organization has learned" section shows the compressed memory (not raw data). 4) Auditor verifies the compression is dynamic (different data → different truths/habits/mistakes).

Constitution score delta

+0.5 (10.0 capped). Memory Compression is the deepest moat — after 4 years, the compressed understanding is impossible to recreate from raw data. It is the accumulated judgment of the organization.

Effort

2 days (1.5 days compression engine + 0.5 day API + frontend)

Build phase

Phase 4 (build after Principles)

Organ #8 — Consciousness (Phase 4, Capstone)

ORGAN #8 Consciousness: always knows where attention/knowledge/trust/conflict is

Purpose

Not consciousness in the UI. Internally. The system always knows: where attention is (which domains are active), where knowledge is (who knows what), where decisions are (what is pending), where trust is (which teams collaborate well), where energy is (which teams are overworked), where uncertainty is (which predictions are low-confidence), where conflicts are (which teams disagree), where learning is (which domains are evolving). This becomes the organizational state — exactly like a brain always knows its own state.

Builds on (existing codebase)

pulse.py tracks organizational temperature/momentum. feed.py tracks the real-time signal feed. All 7 previous organs contribute state. The Consciousness organ composes these into a single "organizational state vector" that is always current. The infrastructure exists — the state-vector composition does not.

Files to create/modify

CREATE `backend/maestro_oem/consciousness.py` — the ConsciousnessEngine. Maintains a real-time organizational state: { "attention": [active domains], "knowledge": [who knows what], "decisions": [pending], "trust": [team pairs + trust score], "energy": [team workload], "uncertainty": [low-confidence predictions], "conflicts": [active contradictions], "learning": [evolving domains] }. Updated on every signal ingest. CREATE GET /api/oem/consciousness endpoint. CREATE `static/js/consciousness.js` — a surface (command-palette only) showing the organizational state as a calm "awareness" view (not a dashboard). MODIFY `static/js/org_dot.js` — the dot now reflects the consciousness state (not just the briefing).

API contract

GET /api/oem/consciousness → returns JSON: { "attention": ["payments", "auth"], "knowledge": {"payments": ["priya.m@acme.com"], "auth": ["carlos.r@acme.com"]}, "decisions": [{"id": "...", "pending_for_days": 3, "domain": "payments"}], "trust": {"engineering-legal": 0.42, "engineering-platform": 0.78}, "energy": {"engineering": 0.85, "legal": 0.30}, "uncertainty": [{"prediction_id": "...", "confidence": 0.18}], "conflicts": [{"contradiction_id": "...", "severity": "high"}], "learning": ["payments", "auth"], "last_updated": "...", "state_summary": "Attention is on payments and auth. Trust between Engineering and Legal is low. One decision has been pending 3 days." }. Must have at least 3 non-empty state dimensions and a state_summary sentence.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/consciousness. Response must have at least 3 non-empty state dimensions (attention, knowledge, decisions, trust, energy, uncertainty, conflicts, learning). 2) Auditor verifies state_summary is a synthesized sentence (not a metric dump). 3) Auditor opens the Consciousness surface via command palette and verifies it renders the state as a calm "awareness" view. 4) Auditor verifies the Organizational Dot now reflects consciousness state (not just briefing). 5) Auditor verifies the state updates on signal ingest (call /api/oem/consciousness, ingest a signal, call again — state should change).

Constitution score delta

+0.5 (10.0 capped). Consciousness is the capstone — it makes Maestro feel alive. The dot, the TODAY brief, the whispers all draw from this state. It is the organizational brain always knowing its own condition.

Effort

3 days (2 days state-vector engine + 0.5 day API + 0.5 day frontend surface + dot integration)

Build phase

Phase 4 (capstone — build last, after all other organs)

Build Order and Dependencies

Phase	Organs	Duration	Why This Order	Unlocks
1	#1 Identity + #2 Curiosity	~4 days	Foundational. Identity provides the self-model. Curiosity provides the questioning. All later organs ref…	Self-awareness + questioning
2	#3 Skepticism + #4 Wisdom	~3.5 days	Skepticism challenges Identity's beliefs. Wisdom synthesizes competing values into judgment. Both de…	Belief challenging + value synthesis
3	#5 Metacognition + #6 Principles	~3.5 days	Metacognition observes the whole. Principles are laws that graduate. Both depend on Wisdom (for jud…	Meta-reflection + long-term wisdom
4	#7 Memory Compression + #8 Consciousness	~5 days	Compression distills everything. Consciousness composes everything. Both depend on all previous org…	Compressed understanding + living state
TOTAL	**8 organs**	**~16 days**	**Cognitive stack: Identity → Curiosity → Skepticism → Wisdom → Metacognition → Pri…**	**10/10 Constitution adherence**

The cognitive stack is the product. Each organ is not a feature — it is a cognitive capability. Together, they form a system that observes (Consciousness), understands (Memory Compression), questions (Curiosity), challenges (Skepticism), judges (Wisdom), reflects (Metacognition), remembers (Principles), and knows itself (Identity). An organization with this stack is not just informed — it is wiser. That is the V4 promise.

The V4 Litmus Test

EVERY INTERACTION MUST LEAVE THE ORGANIZATION SLIGHTLY WISER

The V4 constitutional law, borrowed from the user's vision: "Every interaction with Maestro must leave the organization slightly wiser than it was before." This is the litmus test for every commit, every organ, every API, every UI decision.

What "wiser" means in practice:

• Answering a question is not wisdom. The organization already knew the question. Wisdom is changing how it thinks about the next question.

• Displaying a metric is not wisdom. The organization can read dashboards. Wisdom is explaining why the metric matters and what to do about it.

• Predicting an outcome is not wisdom. Prediction is intelligence. Wisdom is knowing when prediction is unreliable and judgment is needed instead.

• Compressing memory is not wisdom. Compression is efficiency. Wisdom is knowing which memories matter and which to forget.

The V4 acceptance test for every organ: Does this organ change the organization's future judgment? If yes, it is V4. If it only informs without changing judgment, it is V3 — useful but not V4. The 8 organs above all pass this test:

• Identity changes judgment by revealing self-delusion.

• Curiosity changes judgment by opening new questions.

• Skepticism changes judgment by challenging fossilized beliefs.

• Wisdom changes judgment by synthesizing competing values.

• Metacognition changes judgment by revealing systemic blind spots.

• Principles change judgment by encoding long-term wisdom.

• Memory Compression changes judgment by distilling experience into understanding.

• Consciousness changes judgment by making the organization aware of its own state.

If any organ fails this test, redesign it. The V4 bar is not "does it work?" but "does it make the organization wiser?" That is a different standard. Build to it.

The Recurring Pattern — Final Warning

BUILT ≠ APPLIED — 6 OCCURRENCES, DO NOT MAKE IT 7

Across 13 rounds, the "built but not applied" pattern has occurred 6 times:

• Round 7: Added surfaces, did not collapse (claimed 22→4, actually 23).

• Round 8: Built escapeJs, missed 3 handlers.

• Round 10: Built humanize utility, applied to 3 of 24 files.

• Round 12: Built 4 backend engines, wired 0 to frontend.

• Round 13: Wired 3 of 4 engines, silently skipped time-axis.

• Round 13: 3 quality defects from round 12 unfixed (personality value=None, time-axis confidence=None, evolution summary inaccurate).

For V4, the acceptance tests are designed to make this pattern impossible to repeat. Every test has a frontend verification step: "Auditor opens [surface] and verifies [user-visible output]." No organ is "delivered" until the user can see it. The coder must run the FULL acceptance test (API + frontend), not just the API half. If the frontend half fails, the organ is not delivered. Do not claim it. Wire the frontend, then claim it.

The 5-point checklist (unchanged from round 11, still mandatory):

1. Ran the FULL acceptance test (API + frontend)?

2. Checked APPLICATION (frontend calls the API), not EXISTENCE (backend exists)?

3. Ran the FULL test suite (not a subset)?

4. Verified with a LIVE API call?

5. Checked the user-facing UI (opened the surface, saw the output)?

If any answer is "no," the organ is not delivered. The CI pipeline will catch test failures. The acceptance tests will catch built-but-not-applied. The auditor will catch everything else. Do not make it 7.

The V3 → V4 Transition

Before building V4 organs, the coder must close the V3 gaps from round 13. These are small (~4 hours) and must be done first — V4 organs build on V3 engines, so the V3 engines must be fully wired and quality-defect-free.

V3 Gap	Fix	Effort
Time-axis not wired to frontend (Feature #3)	Wire /api/oem/time-axis into TODAY's "one thing learned" item	2 hours
item.sowhat never populated in TODAY (Feature #1)	Fetch /api/oem/sowhat for top recommendation, set item.sowhat	1 hour
Personality value=None for all dimensions (Feature #2)	Populate value 0.0-1.0 in personality.py	30 min
Evolution summary inaccurate (Feature #7)	Fix summary logic to match actual direction counts	30 min
TOTAL V3 cleanup before V4		**~4 hours**

Sequence: V3 cleanup (~4 hours) → V4 Phase 1 (~4 days) → V4 Phase 2 (~3.5 days) → V4 Phase 3 (~3.5 days) → V4 Phase 4 (~5 days). Total: ~16 days + 4 hours. When complete, Constitution adherence is 10/10 and the product is a cognitive system, not a feature collection.

Final Charge to the Coder

V3 was about answering questions. V4 is about changing how the organization thinks. The 8 cognitive organs above are not features — they are cognitive capabilities that, together, form a system whose purpose is to make the organization wiser over time. Every organ builds on existing infrastructure (51 backend modules inspected). Nothing here is vapour. Every organ has an acceptance test that checks both backend AND frontend. Every organ has a build phase and a dependency. The build order is the cognitive stack.

The V4 litmus test for every commit: "Does this leave the organization slightly wiser than it was before?" If yes, ship it. If it only answers a question without changing future judgment, it is V3 — useful but not V4. V4 organs change how the organization thinks. That is the bar.

Do not repeat the pattern. 6 times the coder has built without applying. 6 times the auditor has caught it. The V4 acceptance tests are designed to catch it a 7th time if it happens. Do not make it 7. Wire the frontend. Run the full acceptance test. Check the user-facing UI. Then claim delivery. The CI pipeline will verify the test suite. The acceptance tests will verify the application. The auditor will verify the wisdom.

Build order: V3 cleanup → Phase 1 (Identity + Curiosity) → Phase 2 (Skepticism + Wisdom) → Phase 3 (Metacognition + Principles) → Phase 4 (Memory Compression + Consciousness). ~16 days. When all 8 organs are delivered and user-visible, the product is genuinely a Living Intelligence Layer — not a dashboard, not a copilot, not enterprise software. An organization that has become self-aware, curious, skeptical, wise, reflective, principled, compressed, and conscious. That is the V4 vision. Build it.

The end state is not an enterprise platform. The end state is the accumulated judgment of an organization — impossible to recreate, more valuable than its CRM, more valuable than its project management history, more valuable than its documentation. It answers the question no existing software answers: "Given everything this organization has lived through, how should it think now?" Build toward that destination incrementally. Never rewrite for the sake of the vision. Every architectural improvement must preserve backward compatibility, strengthen the existing OEM, and make the user experience calmer, simpler, and more intelligent than it was before.